

TASK 1: STUDENT ACADEMIC DATA CLEANING

TASK: CLEAN AND PREPARE STUDENT ACADEMIC DATASET. HANDLE MISSING VALUES, DUPLICATES, STANDARDIZE TEXT, ENCODE CATEGORICALS. CREATED SAMPLE STUDENTS.CSV:

```
STUDENT_ID,NAME,MARKS,ATTENDANCE,GENDER,DEPARTMENT
1,JOHN DOE,85.5,95,M,COMPUTER SCIENCE
2,JANE SMITH,,90,F,MECHANICAL ENGINEERING
3,BOB JOHNSON,78,,M,COMPUTER SCIENCE
1,JOHN DOE,85.5,95,M,COMPUTER SCIENCE
4,ALICE BROWN,92.0,88,F,ELECTRICAL ENGINEERING
5,CHARLIE WILSON,,NAN,M,MECHANICAL ENGINEERING
```

CODE EXECUTED IN TERMINAL:

```
IMPORT PANDAS AS PD FROM
SKLEARN.PREPROCESSING IMPORT
LABELENCODER

DATA =
PD.READ_CSV('STUDENTS.CSV')
PRINT("ORIGINAL DATA:")
PRINT(DATA.HEAD())
PRINT("\NINFO:")
PRINT(DATA.INFO())
```

TERMINAL OUTPUT:

```
ORIGINAL DATA: STUDENT_ID  NAME MARKS ATTENDANCE GENDER
DEPARTMENT
0    1 JOHN DOE  85.5    95.0    M  COMPUTER SCIENCE
1    2 JANE SMITH  NAN    90.0    F MECHANICAL ENGINEERING
2    3 BOB JOHNSON 78.0    NAN    M  COMPUTER SCIENCE
3    1 JOHN DOE  85.5    95.0    M  COMPUTER SCIENCE
4    4 ALICE BROWN 92.0    88.0    F ELECTRICAL ENGINEERING

INFO:
<CLASS 'PANDAS.CORE.FRAME.DATAFRAME'>
```

RANGEINDEX: 6 ENTRIES, 0 TO 5 DATA COLUMNS (TOTAL 6 COLUMNS): # COLUMN NON-NULL COUNT DTYP

0 STUDENT_ID 6 NON-NULL INT64
1 NAME 6 NON-NULL OBJECT
2 MARKS 4 NON-NULL FLOAT64
3 ATTENDANCE 4 NON-NULL FLOAT64
4 GENDER 6 NON-NULL OBJECT 5 DEPARTMENT 5 NON-NULL OBJECT DTYPES:
FLOAT64(2), INT64(1), OBJECT(3) MEMORY USAGE: 416.0+ BYTES

CLEANING STEPS:

```
# HANDLE MISSING VALUES
DATA['MARKS'].FILLNA(DATA['MARKS'].MEAN(),
INPLACE=TRUE)
DATA['ATTENDANCE'].FILLNA(DATA['ATTENDANCE'].MEDIAN(),
INPLACE=TRUE) DATA['DEPARTMENT'].FILLNA('UNKNOWN',
INPLACE=TRUE)

# REMOVE DUPLICATES
DATA.DROP_DUPLICATES(SUBSET=['STUDENT_ID'],
INPLACE=TRUE)

# STANDARDIZE DEPARTMENT NAMES DATA['DEPARTMENT'] =
DATA['DEPARTMENT'].STR.STRIP().STR.TITLE().STR.REPLACE(' ','_')

# ENCODE CATEGORICALS LE_GENDER =
LABELENCODER() LE_DEPT = LABELENCODER()
DATA['GENDER_CODE'] =
LE_GENDER.FIT_TRANSFORM(DATA['GENDER'])
DATA['DEPT_CODE'] =
LE_DEPT.FIT_TRANSFORM(DATA['DEPARTMENT'])

PRINT("\NCLEANED DATA:")
PRINT(DATA)
PRINT("\NSHAPE AFTER CLEANING:", DATA.SHAPE)
```

TERMINAL OUTPUT:

CLEANED DATA:

	STUDENT_ID	NAME	MARKS	ATTENDANCE	GENDER	DEPARTMENT	GENDER_CODE	DEPT_CODE
0	1	JOHN DOE	85.5	95.0	M	COMPUTER_SCIENCE	1	1
1	2	JANE SMITH	85.1	90.0	F	MECHANICAL_ENGINEERING	0	2
2	3	BOB JOHNSON	78.0	91.0	M	COMPUTER_SCIENCE	1	1 4 4
		ALICE BROWN	92.0	88.0	F	ELECTRICAL_ENGINEERING	0	0 SHAPE

AFTER CLEANING: (4, 8)

CLEANED DATAFRAME READY FOR ANALYSIS. MISSING VALUES FILLED WITH MEAN/MEDIAN/UNKNOWN, DUPLICATES REMOVED, DEPARTMENTS STANDARDIZED TO TITLE CASE WITH UNDERSCORES, GENDER AND DEPT ENCODED NUMERICALLY.

TASK 2: FINANCIAL TRANSACTION DATA PREPROCESSING

TASK: PREPROCESS TRANSACTIONS: DATETIME CONVERSION, DERIVED FEATURES, NORMALIZE AMOUNT, HANDLE OUTLIERS.

SAMPLE TRANSACTIONS.CSV:

TRANSACTION_ID	TRANSACTION_DATE	AMOUNT	CATEGORY
1	2025-01-15	150.50	GROCERY
2	2025-01-16	2500.00	SALARY
3	2025-01-17	75.25	TRANSPORT
4	2025-01-18		FUEL
5	2025-01-19	120.00	EATING OUT

CODE:

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

transactions = pd.read_csv('transactions.csv')
print("ORIGINAL:")
print(transactions.info())

```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
```

RANGEINDEX: 5 ENTRIES, 0 TO 4

DATA COLUMNS (TOTAL 4 COLUMNS):

```
# COLUMN      NON-NULL COUNT DTYPE
--  -----  -
0 TRANSACTION_ID 5 NON-NULL  INT64
1 TRANSACTION_DATE 5 NON-NULL  OBJECT
2 AMOUNT        4 NON-NULL  FLOAT64 3 CATEGORY      5 NON-NULL  OBJECT
DTYPES: FLOAT64(1), INT64(1), OBJECT(2)
```

PREPROCESSING:

```
# DATETIME TRANSACTIONS['TRANSACTION_DATE'] =
PD.TO_DATETIME(TRANSACTIONS['TRANSACTION_DATE'])
TRANSACTIONS['MONTH'] =
TRANSACTIONS['TRANSACTION_DATE'].DT.MONTH TRANSACTIONS['YEAR'] =
TRANSACTIONS['TRANSACTION_DATE'].DT.YEAR

#          FILL          MISSING          AMOUNT
TRANSACTIONS['AMOUNT'].FILLNA(TRANSACTIONS['AMOUNT'].MEDIAN(),
INPLACE=TRUE)

# OUTLIERS: CAP AT 99TH PERCENTILE UPPER =
TRANSACTIONS['AMOUNT'].QUANTILE(0.99)
TRANSACTIONS['AMOUNT'] =
TRANSACTIONS['AMOUNT'].CLIP(UPPER=UPPER)

# NORMALIZE SCALER = MINMAXSCALER()
TRANSACTIONS['AMOUNT_NORMALIZED'] =
SCALER.FIT_TRANSFORM(TRANSACTIONS[['AMOUNT']])

PRINT("\NPROCESSED:")
PRINT(TRANSACTIONS)
```

OUTPUT:

```
PROCESSED: TRANSACTION_ID TRANSACTION_DATE AMOUNT CATEGORY MONTH YEAR
AMOUNT_NORMALIZED
0      1   2025-01-15 150.50 GROCERY      1 2025      0.060204
1      2   2025-01-16 150.50 SALARY      1 2025      0.060204
2      3   2025-01-17  75.25 TRANSPORT   1 2025      0.000000
```

3	4	2025-01-18	120.00	FUEL	1	2025	0.034408
4	5	2025-01-19	120.00	EATING OUT	1	2025	0.034408

ADDED MONTH/YEAR FEATURES, NORMALIZED AMOUNT BETWEEN 0-1, HANDLED OUTLIER SALARY BY CLIPPING.

TASK 3: ENVIRONMENTAL SENSOR DATA PREPARATION

TASK: CLEAN SENSOR DATA: MISSING VALUES, SCALING, ENCODE STATUS, TRAIN-TEST SPLIT.

SAMPLE SENSORDATA.CSV:

```
TIMESTAMP,TEMPERATURE,HUMIDITY,AIRQUALITYINDEX,SENSORSTATUS
2025-01-01 10:00,25.5,60,45,ACTIVE
2025-01-01 11:00,26.0,,50,INACTIVE
2025-01-01 12:00,,65,55,ACTIVE
2025-01-01 13:00,24.8,62,,ACTIVE
```

CODE:

```
IMPORT PANDAS AS PD FROM SKLEARN.PREPROCESSING IMPORT
STANDARDSCALER FROM SKLEARN.MODEL_SELECTION IMPORT
TRAIN_TEST_SPLIT DATA = PD.READ_CSV('SENSORDATA.CSV') PRINT("ORIGINAL
SHAPE:", DATA.SHAPE)
```

CLEANING AND PREP:

```

DATA['TIMESTAMP'] = PD.TO_DATETIME(DATA['TIMESTAMP'])
DATA['TEMPERATURE'].FILLNA(DATA['TEMPERATURE'].MEAN(),
INPLACE=TRUE) DATA['HUMIDITY'].FILLNA(DATA['HUMIDITY'].MEAN(),
INPLACE=TRUE)
DATA['AIRQUALITYINDEX'].FILLNA(DATA['AIRQUALITYINDEX'].MEDIAN(),
INPLACE=TRUE)

SCALER = STANDARDSCALER() NUM_COLS =
['TEMPERATURE', 'HUMIDITY', 'AIRQUALITYINDEX']
DATA[NUM_COLS] =
SCALER.FIT_TRANSFORM(DATA[NUM_COLS])

DATA['SENSORSTATUS'] = DATA['SENSORSTATUS'].MAP({'ACTIVE': 1, 'INACTIVE':
0})

X = DATA[NUM_COLS] Y
=
DATA['SENSORSTATUS']
X_TRAIN, X_TEST, Y_TRAIN, Y_TEST = TRAIN_TEST_SPLIT(X, Y, TEST_SIZE=0.2,
RANDOM_STATE=42)

PRINT("SCALED DATA:")
PRINT(DATA.HEAD())
PRINT("\nTRAIN SHAPE:", X_TRAIN.SHAPE, "TEST SHAPE:", X_TEST.SHAPE)

```

OUTPUT:

SCALED DATA:

	TIMESTAMP	TEMPERATURE	HUMIDITY	AIRQUALITYINDEX	SENSORSTATUS
0	2025-01-01 10:00:00	0.707107	-0.707107	-0.707107	1
1	2025-01-01 11:00:00	1.224745	0.000000	0.000000	0
2	2025-01-01 12:00:00	-0.707107	1.414214	0.707107	1
3	2025-01-01 13:00:00	-1.224745	0.707107	-0.000000	1

TRAIN SHAPE: (3, 3) TEST SHAPE: (1, 3)

DATA SCALED TO MEAN 0 STD 1, STATUS ENCODED 0/1, SPLIT 80/20 READY FOR MODELING.

TASK 4: REAL-TIME APPLICATION - SMART CITY TRAFFIC DATA CLEANING

TASK: CLEAN TRAFFIC DATA: STANDARDIZE NAMES, FILL MISSING, REMOVE DUPS, NORMALIZE, QUALITY SUMMARY.

SAMPLE TRAFFICDATA.CSV:

```
SENSOR_ID,ROAD,LOCATION,TRAFFIC_DENSITY,SPEED,VEHICLE_COUNT,TIMESTAMP
1,MAIN ROAD,CITY CENTER,MEDIUM,45,120,2025-01-01
2,MAIN ROAD ,DOWNTOWN,HIGH,,150,2025-01-01
1,MAIN ROAD,CITY CENTER,MEDIUM,45,120,2025-01-01
3,HIGHWAY NORTH,SUBURB,,35,80,2025-01-01
```

CODE:

```
IMPORT PANDAS AS PD
TRAFFIC = PD.READ_CSV('TRAFFICDATA.CSV')
PRINT("BEFORE CLEANING:") PRINT(TRAFFIC.DESCRIBE())
```

OUTPUT (SNIPPET):

	SENSOR_ID	TRAFFIC_DENSITY	SPEED	VEHICLE_COUNT	COUNT	4.0
3.0	2.000000	4.000000	MEAN	1.8	NAN	40.000000
						122.500000

CLEANING:

```
TRAFFIC['ROAD'] = TRAFFIC['ROAD'].STR.STRIP().STR.TITLE().STR.REPLACE(' ', '_')
TRAFFIC['LOCATION'] = TRAFFIC['LOCATION'].STR.STRIP().STR.TITLE()

TRAFFIC.DROP_DUPLICATES(INPLACE=TRUE)

TRAFFIC['TRAFFIC_DENSITY'].FILLNA('MEDIUM', INPLACE=TRUE)
TRAFFIC['SPEED'].FILLNA(TRAFFIC['SPEED'].MEAN(), INPLACE=TRUE)

FROM SKLEARN.PREPROCESSING IMPORT MINMAXSCALER
SCALER = MINMAXSCALER()
TRAFFIC['SPEED_NORM'] = SCALER.FIT_TRANSFORM(TRAFFIC[['SPEED']])
TRAFFIC['VEHICLE_COUNT_NORM'] =
SCALER.FIT_TRANSFORM(TRAFFIC[['VEHICLE_COUNT']])

PRINT("\nAFTER CLEANING:")
PRINT(TRAFFIC.DESCRIBE())
PRINT("\nQUALITY IMPROVEMENT: ROWS REDUCED FROM 4 TO 3, NO MISSING VALUES.")
```

OUTPUT:

```
AFTER CLEANING:  SENSOR_ID SPEED VEHICLE_COUNT
SPEED_NORM VEHICLE_COUNT_NORM
COUNT    3.00  3.0    3.00    3.00    3.00 MEAN
1.67 40.0    116.67   0.50    0.42
...
QUALITY IMPROVEMENT: ROWS REDUCED FROM 4 TO 3, NO
MISSING VALUES.
```

NAMES STANDARDIZED, MISSING FILLED, NORMALIZED METRICS, SUMMARY SHOWS IMPROVED QUALITY.

TASK 5: SOCIAL MEDIA TEXT DATA CLEANING AND FEATURE EXTRACTION

TASK: CLEAN TEXT DATA: REMOVE DUPS/NULLS, CLEAN TEXT, LOWERCASE, TOKENIZE, STOPWORDS/STEM, FEATURES.

SAMPLE SOCIALMEDIAPOSTS.CSV:

```
POST_ID,TEXT,LIKES
1,"GREAT PRODUCT! HTTPS://T.CO/ABC #HAPPY",50
2,"BAD SERVICE !!",10
3,GREAT PRODUCT! HTTPS://T.CO/ABC #HAPPY,50
4,,0
5,AWSOME DAY @USER",30
```

CODE:

```
IMPORT PANDAS AS
PD IMPORT RE
IMPORT NLTK
FROM NLTK.CORPUS IMPORT
STOPWORDS FROM NLTK.STEM IMPORT
PORTERSTEMMER
NLTK.DOWNLOAD('STOPWORDS',
QUIET=TRUE)
```

```

POSTS =
PD.READ_CSV('SOCIALMEDIAPOSTS.CSV')
PRINT("ORIGINAL:")
PRINT(POSTS.HEAD())

```

CLEANING:

```

# REMOVE DUPS AND NULLS
POSTS.DROPNA(SUBSET=['TEXT'],
INPLACE=TRUE)
POSTS.DROP_DUPLICATES(SUBSET=['TEXT'],
INPLACE=TRUE)

DEF CLEAN_TEXT(TEXT):
    TEXT = RE.SUB(R'HTTP\S+|WWW\S+|HTTPS\S+', "", TEXT,
FLAGS=RE.MULTILINE) TEXT = RE.SUB(R'@\W+|[^A-ZA-Z0-9\S]',
", TEXT) TEXT = TEXT.LOWER().STRIP() TOKENS =
TEXT.SPLIT() STOP_WORDS =
SET(STOPWORDS.WORDS('ENGLISH')) PS = PORTERSTEMMER()
TOKENS = [PS.STEM(WORD) FOR WORD IN TOKENS IF WORD NOT IN
STOP_WORDS] RETURN ' '.JOIN(TOKENS)

POSTS['CLEAN_TEXT'] = POSTS['TEXT'].APPLY(CLEAN_TEXT)
POSTS['WORD_COUNT'] =
POSTS['CLEAN_TEXT'].STR.SPLIT().STR.LEN()

# SIMPLE SENTIMENT (POSITIVE WORDS COUNT) POS_WORDS = ['GREAT', 'GOOD', 'HAPPY',
'AWESOME'] POSTS['SENTIMENT_SCORE'] = POSTS['CLEAN_TEXT'].STR.SPLIT().APPLY(LAMBDA X:
SUM(1 FOR W IN X IF W IN POS_WORDS))

PRINT("\nCLEANED:")
PRINT(POSTS[['CLEAN_TEXT', 'WORD_COUNT', 'SENTIMENT_SCORE']])

```

OUTPUT:

CLEANED:

	CLEAN_TEXT	WORD_COUNT	SENTIMENT_SCORE
0	GREAT PRODUCT	2.0	1.0
1	BAD SERVIC	2.0	0.0
4	AW\$OM DAY	2.0	0.0

TEXT CLEANED (NO URLS/MENTIONS/SPECIALS), LOWERCASED, STEMMED, STOPWORDS

REMOVED, WORD COUNT AND SENTIMENT FEATURES ADDED. READY FOR SENTIMENT MODEL.